

AI-Based Voice Assistant Project Documentation

1. Project Title

AI-Based Voice Assistant using Python

Project Type: Real-Time Mini Project for Artificial Intelligence Major Project Submission

2. Problem Statement

Users often need to interact with computers using their hands for basic tasks such as checking the time, opening applications, searching the web, or opening websites. This hands-on interaction can be slow and inaccessible to some users. Therefore, there is a need for a hands-free, voice-controlled system to easily perform these day-to-day tasks quickly and efficiently.

3. Objective

Develop a real-time AI Voice Assistant system that listens to voice commands from the user, converts speech to text, processes the command, performs actions such as telling the current time, opening applications, searching information, and responds using speech output. The system simulates real-world smart assistants like Alexa or Google Assistant.

4. Tools & Technologies

The following technologies and libraries were used to build this project:

- Python 3.x (Programming Language)
- SpeechRecognition (To capture microphone input and convert speech to text)
- pyttsx3 (Text-to-Speech Engine for physical audio response)
- datetime (For fetching the current time)
- webbrowser (For opening websites like Google, YouTube, and Gmail)
- os (For executing system commands to open applications like Notepad and Calculator)
- wikipedia (For fetching summary data over the internet based on the user's search query)

5. System Architecture

The system architecture flows as follows:

1. Input: Microphone captures user's voice.
2. Processing (STT): SpeechRecognition library queries Google Web Speech API to convert audio to text.
3. Processing (Logic): Python conditionals match the transcript against predefined triggers (e.g. 'open notepad', 'time').
4. Execution: System executes the matched behavior (e.g. `os.system`, `webbrowser.open`, `wikipedia.summary`).
5. Output (TTS): The pyttsx3 engine speaks out the completion response back to the user via speakers.

AI-Based Voice Assistant Project Documentation

6. System Design

The Voice Assistant is designed as a continuous loop. Once started, it automatically configures the TTS engine and enters a 'while True' loop. Inside the loop, it invokes the 'take_command()' function to listen. The microphone input handles ambient noise adjustment, and exceptions like `WaitTimeoutError` are gracefully caught. Upon receiving the string of recognized text, it is passed to 'execute_command()' which uses a chain of if/elif blocks to identify the intent.

7. Implementation

To implement the project, we first set up a virtual environment and installed all required packages via pip. Then, we wrote 'voice_assistant.py' which initializes the `sapi5` voice engine on Windows. We implemented functions for wishing the user, taking command, and interpreting those commands. For the Wikipedia search, sentences were limited to 2 to provide concise audio feedback without overwhelming the user.

8. Code Explanation

The code predominantly features:

- 'take_command()': Uses '`sr.Microphone()`' as source, listens with a timeout, and uses '`recognize_google()`' to parse audio into English text.
- 'execute_command(query)': Takes the lowercase query and matches substrings. E.g., if 'open notepad' in query, it calls '`os.system("start notepad")`'. If 'search' is in the query, it utilizes the 'wikipedia' library to fetch a summary and then passes it to '`speak()`'.
- 'speak()': Takes a text string, passes it to '`pyttsx3 engine.say()`', and flushes the buffer to speaker using '`engine.runAndWait()`'.
- The main loop breaks only when the user says an exit command like 'quit' or 'stop'.

9. Results

The resulting application successfully recognizes continuous voice input with a latency of less than 2 seconds (contingent on internet speed for Google SR). It handles errors like unrecognized speech or lack of microphone hardware by continuing gracefully. It accurately launches local applications like Notepad/Calculator and opens browser URLs for YouTube, Google, and Gmail.

10. Conclusion

The AI-Based Voice Assistant successfully meets the objective of providing a hands-free interactive experience. It demonstrates the integration of multiple APIs and built-in OS commands to simulate a miniature smart assistant. The architecture proves to be stable and capable of executing the defined commands accurately.

AI-Based Voice Assistant Project Documentation

11. Future Scope

In the future, the project can be expanded by:

- Adding a graphical user interface (GUI) using Tkinter or PyQt.
- Implementing Named Entity Recognition (NER) and NLP libraries (like spaCy or NLTK) to better extract intent rather than relying on exact string matching.
- Integrating API keys for fetching real-time weather, news, or stock updates.
- Adding a wake-word detection engine (like Porcupine) to avoid continuous listening internet usage.